

Design Patterns in My Laravel Project

Subscription Box Application

1. Project Overview

The subscription_box project is a Laravel web application for a sports subscription box service. It supports customer registration, login, subscription plans, box browsing, cart and order handling, rewards, and admin management of plans, inventory, themes, orders, returns, and reward accounts.

The main architectural pattern used is Model-View-Controller (MVC), supported by several Laravel framework patterns:

- Front Controller
- Router / Controller Dispatch
- Active Record through Eloquent Models
- Blade Templating
- Facades
- Dependency Injection

2. The Main Pattern: MVC

What is MVC?

MVC stands for Model - View - Controller. It splits the application into 3 parts so each part has one job.

Part	What It Does	In My Project
Model	Handles the database	app/Models (e.g. User, Box, Plan)
View	Shows the page to the user	resources/views (Blade files)
Controller	Connects Model and View	app/Http/Controllers

Simple Example

When a customer visits /boxes, this is what happens:

```
1. Browser sends request to /boxes
2. routes/web.php sends it to CustomerController
3. CustomerController gets boxes from the Box model
4. boxes.blade.php shows the boxes on screen
```

3. Other Patterns Used

Front Controller

All requests go through one file first: public/index.php

This is like a security guard at the door — every visitor must pass through it before going anywhere in the app.

Router Pattern

Routes are defined in routes/web.php. The route file maps URLs to controller methods or small view closures:

```
Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard');
Route::post('/select-plan', [CustomerController::class, 'selectPlan'])->name('select.plan');
Route::get('/admin/dashboard', [AdminController::class, 'dashboard'])->name('admin.dashboard');
```

Facade Pattern

Laravel lets you use services in a simple way using Facades. Instead of setting up complex objects, you just call them directly.

Facade	Usage
Auth	Login, logout, current authenticated user, guards.
Hash	Password hashing and checking.
DB	Database transactions.
Schema	Runtime checks for tables or optional columns.
Route	Route definitions.

```
Auth::user();           // get the logged in user
Hash::make($pass);      // encrypt a password
DB::transaction();       // run a safe database operation
```

Active Record (Eloquent)

Each database table has a Model class. The model can save, find, and delete records by itself.

Laravel Eloquent models use the Active Record pattern. Each model class represents a database table and contains methods for relationships and persistence.

Model	Table	Purpose
User	users	Customer authentication identity.
Admin	admins	Admin authentication identity.
Customer	customers	Customer profile, preferences, address, and selected plan.
Plan	plans	Subscription plan details.
Subscription	subscriptions	User subscription status and billing dates.
Box	boxes	Subscription box catalog item.
BoxOrder	box_orders	Customer box order/cart entry.
InventoryItem	inventory_items	Admin-managed inventory.
RewardAccount	reward_accounts	User reward balance and tier.
RewardTransaction	reward_transactions	Reward earning/redemption history.
Returns	returns	Return requests.

```
// Find a plan from the database
Plan::find($id);

// Create a new subscription
Subscription::updateOrCreate([...]);
```

Dependency Injection

Instead of creating objects manually, Laravel passes them in automatically. This is called Dependency Injection.

```
// Laravel automatically gives us $request and $adminController
public function confirmShipping(Request $request, AdminController $adminController)
{
    // use them here
}
```

4. Where Each Pattern Appears

Pattern	Where in the Project
MVC	Models in app/Models, Views in resources/views, Controllers in app/Http/Controllers
Front Controller	public/index.php
Facade	Auth, Hash, DB used inside controllers
Active Record	All Eloquent models (User, Box, Plan, etc.)
Dependency Injection	Controller method parameters (Request, AdminController)

5. Controllers

5.1 Authentication Controllers

Location: app/Http/Controllers/Auth

classes:

- RegisterController — displays registration page, validates input, creates customer/admin accounts.
- LoginController — displays login page, separates login behavior for admins and customers.

5.2 CustomerController

Location: app/Http/Controllers/CustomerController.php

Responsibilities:

- Select, pause, and resume subscriptions.
- Display boxes and customize them.
- Add boxes to cart and confirm shipping.
- Swap boxes and redeem rewards.

Important methods:

<code>selectPlan()</code>	<code>pauseSubscription()</code>	<code>resumeSubscription()</code>
<code>boxes()</code>	<code>customize()</code>	<code>addToCart()</code>
<code>saveCustomizedBox()</code>	<code>cart()</code>	<code>swapBox()</code>
<code>confirmShipping()</code>	<code>reward()</code>	<code>redeemReward()</code>

5.3 DashboardController

Location: app/Http/Controllers/DashboardController.php

Responsibilities:

- Load the authenticated user's customer profile, subscription, and plan data.
- Load all box orders and compute upcoming orders.
- Compute valid box swap options and return the customer dashboard view.

5.4 AdminController

Location: app/Http/Controllers/AdminController.php

Responsibilities:

- Load admin dashboard data.
- Manage plans, inventory, and monthly themes.
- batch order s and handle returns.
- Award and redeem reward points.

6. Summary

My project uses the following design patterns:

- MVC — the main structure of the whole app
- Front Controller — all requests go through one entry point
- Facade — easy access to Laravel services like Auth and DB
- Active Record — each model manages its own database table
- Dependency Injection — Laravel passes objects into methods automatically

These patterns are all provided by the Laravel framework. By using Laravel, my project automatically follows good software design practices.